

FLASHATTENTION-2: FASTER ATTENTION WITH BETTER PARALLELISM AND WORK PARTITIONING

ABSTRACT

Scaling Transformers to longer sequence lengths has been a major problem in the last several years, promising to improve performance in language modeling and high-resolution image understanding, as well as to unlock new applications in code, audio, and video generation.

The attention layer is the main bottleneck in scaling to longer sequences, as its runtime and memory increase quadratically in the sequence length.

FLASHATTENTION (Dao et al., 2022) exploits the asymmetric GPU memory hierarchy to bring significant memory saving (linear instead of quadratic) and runtime speedup (2-4x compared to optimized baselines), with no approximation.

However, FLASHATTENTION is still not nearly as fast as optimized matrix-multiply (GEMM) operations, reaching only 25-40% of the theoretical maximum FLOPs/s. We observe that the inefficiency is due to suboptimal work partitioning between different thread blocks and warps on the GPU, causing either low-occupancy or unnecessary shared memory reads/writes.

We propose FLASHATTENTION-2, with better work partitioning to address these issues. In particular, we (1) tweak the algorithm to reduce the number of non-matmul FLOPs (2) parallelize the attention computation, even for a single head, across different thread blocks to increase occupancy, and (3) within each thread block, distribute the work between warps to reduce communication through shared memory. These yield around 2x speedup compared to FLASHATTENTION, reaching 50-73% of the theoretical maximum FLOPs/s on A100 and getting close to the efficiency of GEMM operations.

1 INTRODUCTION

Dao et al. (2022) proposed to reorder the attention computation and leverages classical techniques (tiling, recomputation) to significantly speed it up and reduce memory usage from quadratic to linear in sequence length. This yields 2-4 $\times$ wall-clock time speedup over optimized baselines, up to 10-20 $\times$ memory saving, with no approximation, and as a result FLASHATTENTION has seen wide adoption in large-scale training and inference of Transformers

while FLASHATTENTION is already 2-4 $\times$ faster than a standard attention implementation, the forward pass only reaches 30-50% of the theoretical maximum FLOPs/s of the device (Fig. 6), while the backward pass is even more challenging, reaching only 25-35% of maximum throughput on A100 GPU (Fig. 7). In contrast, optimized GEMM can reach up to 80-90% of the theoretical maximum device throughput.

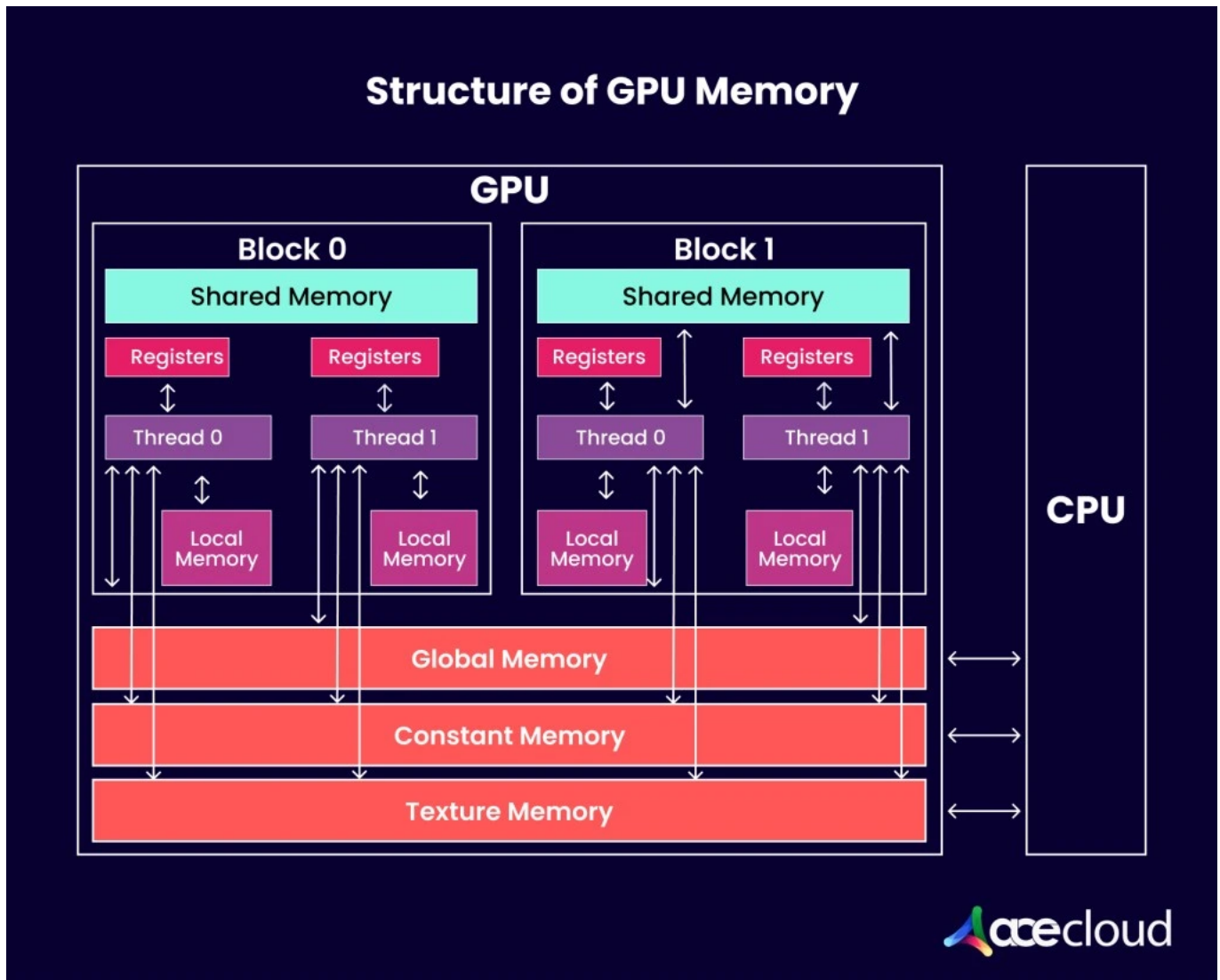
Building on FLASHATTENTION, we propose FLASHATTENTION-2 with better parallelism and work partitioning to address these challenges.

1. In Section 3.1, we tweak the algorithms to reduce the number of non-matmul FLOPs while not changing the output. While the non-matmul FLOPs only account for a small fraction of the total FLOPs, they take longer to perform as GPUs have specialized units for matrix multiply, and as a result the matmul throughput can be up to 16 $\times$ higher than non-matmul throughput. It is thus important to reduce non-matmul FLOPs and spend as much time as possible doing matmul FLOPs.

2. We propose to parallelize both the forward pass and backward pass along the sequence length dimension, in addition to the batch and number of heads dimension. This increases occupancy (utilization of GPU resources) in the case where the sequences are long (and hence batch size is often small)
3. Even within one block of attention computation, we partition the work between different warps of a thread block to reduce communication and shared memory reads/writes.

2 BACKGROUND We provide some background on the performance characteristics and execution model of GPUs. We also describe the standard implementation of attention, as well as FLASHATTENTION

2.1 HARDWARE CHARACTERISTICS GPU performance characteristics. The GPU consists of compute elements (e.g., floating point arithmetic units) and a memory hierarchy. Most modern GPUs contain specialized units to accelerate matrix multiply in low-precision (e.g., Tensor Cores on Nvidia GPUs for FP16/BF16 matrix multiply). The memory hierarchy comprise of high bandwidth memory (HBM), and on-chip SRAM (aka shared memory). As an example, the A100 GPU has 40-80GB of high bandwidth memory (HBM) with bandwidth 1.5-2.0TB/s and 192KB of on-chip SRAM per each of 108 streaming multiprocessors with bandwidth estimated around 19TB/s (Jia et al., 2018; Jia and Van Sandt, 2021). As the L2 cache is not directly controllable by the programmer, we focus on the HBM and SRAM for the purpose of this discussion.



Execution Model. GPUs have a massive number of threads to execute an operation (called a kernel). Threads are organized into thread blocks, which are scheduled to run on streaming multiprocessors (SMs). Within each thread blocks, threads are grouped into warps (a group of 32 threads). Threads within a warp can

communicate by fast shuffle instructions or cooperate to perform matrix multiply. Warps within a thread block can communicate by reading from / writing to shared memory. Each kernel loads inputs from HBM to registers and SRAM, computes, then writes outputs to HBM.

2.2 STANDARD ATTENTION IMPLEMENTATION

Given input sequences $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ where N is the sequence length and d is the head dimension, we want to compute the attention output $\mathbf{O} \in \mathbb{R}^{N \times d}$:

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^T \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \text{softmax}(\mathbf{S}) \in \mathbb{R}^{N \times N}, \quad \mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N \times d},$$

where softmax is applied row-wise.¹ For multi-head attention (MHA), this same computation is performed in parallel across many heads, and parallel over the batch dimension (number of input sequences in a batch).

The backward pass of attention proceeds as follows. Let $d\mathbf{O} \in \mathbb{R}^{N \times d}$ be the gradient of $\mathbf{O}$ with respect to some loss function. Then by the chain rule (aka backpropagation):

$$\begin{aligned} d\mathbf{V} &= \mathbf{P}^T d\mathbf{O} \in \mathbb{R}^{N \times d} & d\mathbf{P} &= d\mathbf{O}\mathbf{V}^T \in \mathbb{R}^{N \times N} \\ d\mathbf{S} &= d\text{softmax}(d\mathbf{P}) \in \mathbb{R}^{N \times N} & d\mathbf{Q} &= d\mathbf{S}\mathbf{K} \in \mathbb{R}^{N \times d} & d\mathbf{K} &= d\mathbf{S}^T \mathbf{Q} \in \mathbb{R}^{N \times d}, \end{aligned}$$

where $d\text{softmax}$ is the gradient (backward pass) of softmax applied row-wise. One can work out that if $p = \text{softmax}(s)$ for some vector s and p , then with output gradient dp , the input gradient $ds = (\text{diag}(p) - pp^T)dp$.

Standard attention implementations materialize the matrices $\mathbf{S}$ and $\mathbf{P}$ to HBM, which takes $O(N^2)$ memory. Often $N \gg d$ (typically N is on the order of 1k–8k and d is around 64–128). The standard attention implementation (1) calls the matrix multiply (GEMM) subroutine to multiply $\mathbf{S} = \mathbf{Q}\mathbf{K}^T$, writes the result to HBM, then (2) loads $\mathbf{S}$ from HBM to compute softmax and write the result $\mathbf{P}$ to HBM, and finally (3) calls GEMM to get $\mathbf{O} = \mathbf{P}\mathbf{V}$. As most of the operations are bounded by memory bandwidth, the large number of memory accesses translates to slow wall-clock time. Moreover, the required memory is $O(N^2)$ due to having to materialize $\mathbf{S}$ and $\mathbf{P}$. Moreover, one has to save $\mathbf{P} \in \mathbb{R}^{N \times N}$ for the backward pass to compute the gradients.

2.3 FLASHATTENTION To speed up attention on hardware accelerators such as GPU, (Dao et al., 2022) proposes an algorithm to reduce the memory reads/writes while maintaining the same output (without approximation)

2.3.1 FORWARD PASS

FLASHATTENTION applies the classical technique of tiling to reduce memory IOs, by (1) loading blocks of inputs from HBM to SRAM, (2) computing attention with respect to that block, and then (3) updating the output without writing the large intermediate matrices $\mathbf{S}$ and $\mathbf{P}$ to HBM.

As the softmax couples entire rows or blocks of row, online softmax (Milakov and Gimelshein, 2018; Rabe and Staats, 2021) can split the attention computation into blocks, and rescale the output of each block to finally get the right result (with no approximation).

By significantly reducing the amount of memory reads/writes, FLASHATTENTION yields 2-4x wall-clock speedup over optimized baseline attention implementations.

We describe the online softmax technique (Milakov and Gimelshein, 2018) and how it is used in attention (Rabe and Staats, 2021). For simplicity, consider just one row block of the attention matrix S , of the form $\begin{bmatrix} S^{(1)} & S^{(2)} \end{bmatrix}$ for some matrices $S^{(1)}, S^{(2)} \in \mathbb{R}^{B_r \times B_c}$, where B_r and B_c are the row and column block sizes. We want to compute softmax of this row block and multiply with the value, of the form $\begin{bmatrix} V^{(1)} \\ V^{(2)} \end{bmatrix}$ for some matrices $V^{(1)}, V^{(2)} \in \mathbb{R}^{B_c \times d}$. Standard softmax would compute:

$$m = \max(\text{rowmax}(S^{(1)}), \text{rowmax}(S^{(2)})) \in \mathbb{R}^{B_r} \quad \ell = \text{rowsum}(e^{S^{(1)}-m}) + \text{rowsum}(e^{S^{(2)}-m}) \in \mathbb{R}^{B_r}$$

$$P = \begin{bmatrix} P^{(1)} & P^{(2)} \end{bmatrix} = \text{diag}(\ell)^{-1} \begin{bmatrix} e^{S^{(1)}-m} & e^{S^{(2)}-m} \end{bmatrix} \in \mathbb{R}^{B_r \times 2B_c}$$

$$O = \begin{bmatrix} P^{(1)} & P^{(2)} \end{bmatrix} \begin{bmatrix} V^{(1)} \\ V^{(2)} \end{bmatrix} = \text{diag}(\ell)^{-1} e^{S^{(1)}-m} V^{(1)} + e^{S^{(2)}-m} V^{(2)} \in \mathbb{R}^{B_r \times d}.$$

Online softmax instead computes “local” softmax with respect to each block and rescale to get the right output at the end:

$$m^{(1)} = \text{rowmax}(S^{(1)}) \in \mathbb{R}^{B_r} \quad \ell^{(1)} = \text{rowsum}(e^{S^{(1)}-m^{(1)}}) \in \mathbb{R}^{B_r}$$

$$\tilde{P}^{(1)} = \text{diag}(\ell^{(1)})^{-1} e^{S^{(1)}-m^{(1)}} \in \mathbb{R}^{B_r \times B_c} \quad O^{(1)} = \tilde{P}^{(1)} V^{(1)} = \text{diag}(\ell^{(1)})^{-1} e^{S^{(1)}-m^{(1)}} V^{(1)} \in \mathbb{R}^{B_r \times d}$$

$$m^{(2)} = \max(m^{(1)}, \text{rowmax}(S^{(2)})) = m$$

$$\ell^{(2)} = e^{m^{(1)}-m^{(2)}} \ell^{(1)} + \text{rowsum}(e^{S^{(2)}-m^{(2)}}) = \text{rowsum}(e^{S^{(1)}-m}) + \text{rowsum}(e^{S^{(2)}-m}) = \ell$$

$$\tilde{P}^{(2)} = \text{diag}(\ell^{(2)})^{-1} e^{S^{(2)}-m^{(2)}}$$

$$O^{(2)} = \text{diag}(\ell^{(1)}/\ell^{(2)}) O^{(1)} + \tilde{P}^{(2)} V^{(2)} = \text{diag}(\ell^{(2)})^{-1} e^{S^{(1)}-m} V^{(1)} + \text{diag}(\ell^{(2)})^{-1} e^{S^{(2)}-m} V^{(2)} = O.$$

We show how FLASHATTENTION uses online softmax to enable tiling (Fig. 1) to reduce memory reads/writes.

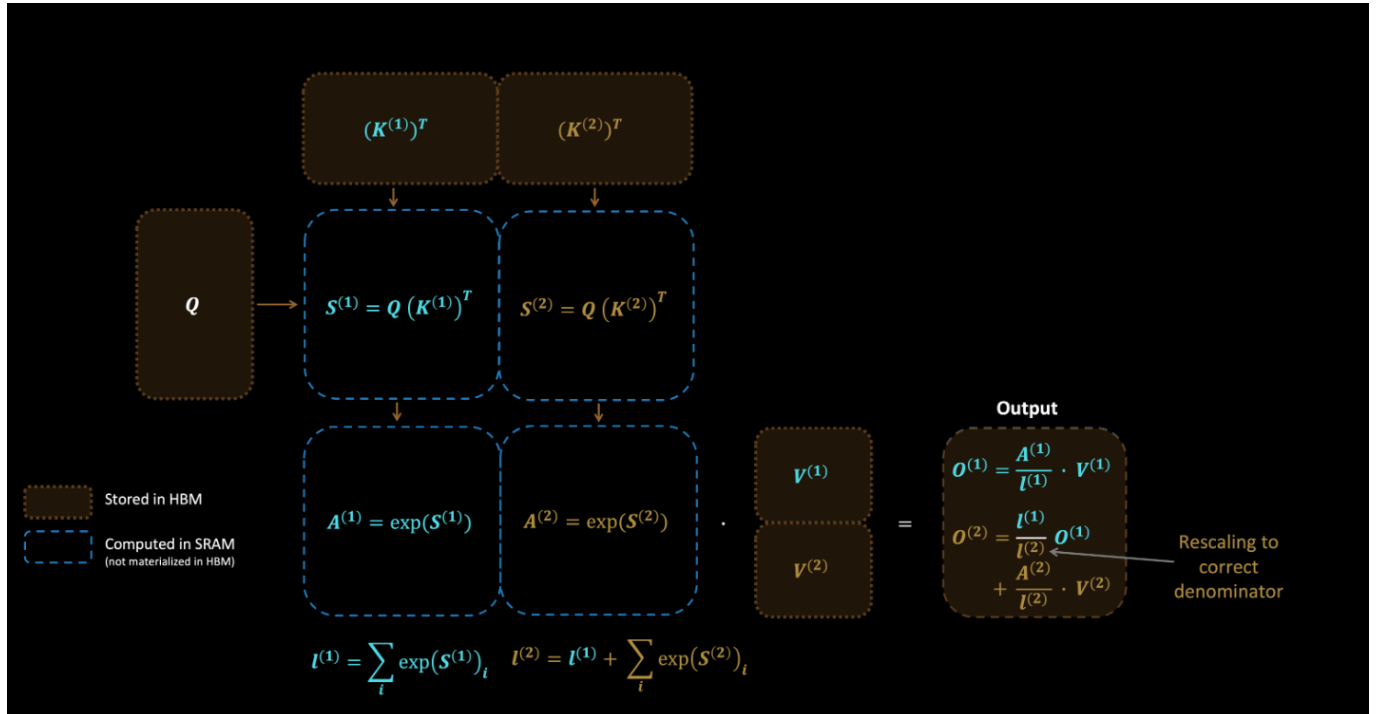


Figure 1: Diagram of how FLASHATTENTION forward pass is performed, when the key K is partitioned into two blocks and the value V is also partitioned into two blocks. By computing attention with respect to each block and rescaling the output, we get the right answer at the end, while avoiding expensive memory reads/writes of the intermediate matrices S and P . We simplify the diagram, omitting the step in softmax that subtracts each element by the row-wise max.

2.3.2 BACKWARD PASS

In the backward pass, by re-computing the values of the attention matrices S and P once blocks of inputs Q , K , V are already loaded to SRAM, FLASHATTENTION avoids having to store large intermediate values. By not having to save the large matrices S and P of size $N \times N$, FLASHATTENTION yields 10-20x memory saving depending on sequence length (memory required is linear in sequence length N instead of quadratic). The backward pass also achieves 2-4x wall-clock speedup due to reduced memory reads/writes.

The backward pass applies tiling to the equations in Section 2.2. Though the backward pass is simpler than the forward pass conceptually (there is no softmax rescaling), the implementation is significantly more involved. This is because there are more values to be kept in SRAM to perform 5 matrix multiplies in the backward pass, compared to just 2 matrix multiplies in the forward pass.

3 FLASHATTENTION-2: ALGORITHM, PARALLELISM, AND WORK PARTITIONING

We describe the FLASHATTENTION-2 algorithm, which includes several tweaks to FLASHATTENTION to reduce the number of non-matmul FLOPs. We then describe how to parallelize the computation on different thread blocks to make full use of the GPU resources. Finally we describe how we partition the work between different warps within one thread block to reduce the amount of shared memory access. These improvements lead to 2-3x speedup as validated in Section 4.

3.1 ALGORITHM

---repetition---

We tweak the algorithm from FLASHATTENTION to reduce the number of non-matmul FLOPs. This is because modern GPUs have specialized compute units (e.g., Tensor Cores on Nvidia GPUs) that make matmul much faster. As an example, the A100 GPU has a max theoretical throughput of 312 TFLOPs/s of FP16/BF16 matmul, but only 19.5 TFLOPs/s of non-matmul FP32. Another way to think about this is that each non-matmul FLOP is 16x more expensive than a matmul FLOP. To maintain high throughput (e.g., more than 50% of the maximum theoretical TFLOPs/s), we want to spend as much time on matmul FLOPs as possible

3.1.1 FORWARD PASS(To revisit)

We revisit the online softmax trick as shown in Section 2.3 and make two minor tweaks to reduce non-matmul FLOPs

1. We do not have to rescale both terms of the output update by $\text{diag}(\ell^{(2)})^{-1}$:

$$\mathbf{O}^{(2)} = \text{diag}(\ell^{(1)} / \ell^{(2)}) \mathbf{O}^{(1)} + \text{diag}(\ell^{(2)})^{-1} e^{S^{(2)} - m^{(2)}} \mathbf{V}^{(2)}.$$

We can instead maintain an “un-scaled” version of $\mathbf{O}^{(2)}$ and keep around the statistics $\ell^{(2)}$:

$$\tilde{\mathbf{O}}^{(2)} = \text{diag}(\ell^{(1)})^{-1} \mathbf{O}^{(1)} + e^{S^{(2)} - m^{(2)}} \mathbf{V}^{(2)}.$$

Only at the very end of the loop do we scale the final $\tilde{\mathbf{O}}^{(\text{last})}$ by $\text{diag}(\ell^{(\text{last})})^{-1}$ to get the right output.

2. We do not have to save both the max $m^{(j)}$ and the sum of exponentials $\ell^{(j)}$ for the backward pass. We only need to store the logsumexp $L^{(j)} = m^{(j)} + \log(\ell^{(j)})$.

In the simple case of 2 blocks in Section 2.3, the online softmax trick now becomes:

$$\begin{aligned}
 m^{(1)} &= \text{rowmax}(\mathbf{S}^{(1)}) \in \mathbb{R}^{B_r} & \ell^{(1)} &= \text{rowsum}(e^{\mathbf{S}^{(1)} - m^{(1)}}) \in \mathbb{R}^{B_r} \\
 \tilde{\mathbf{O}}^{(1)} &= e^{\mathbf{S}^{(1)} - m^{(1)}} \mathbf{V}^{(1)} \in \mathbb{R}^{B_r \times d} & m^{(2)} &= \max(m^{(1)}, \text{rowmax}(\mathbf{S}^{(2)})) = m \\
 \ell^{(2)} &= e^{m^{(1)} - m^{(2)}} \ell^{(1)} + \text{rowsum}(e^{\mathbf{S}^{(2)} - m^{(2)}}) = \text{rowsum}(e^{\mathbf{S}^{(1)} - m} + e^{\mathbf{S}^{(2)} - m}) = \ell \\
 \tilde{\mathbf{P}}^{(2)} &= \text{diag}(\ell^{(2)})^{-1} e^{\mathbf{S}^{(2)} - m^{(2)}} \\
 \tilde{\mathbf{O}}^{(2)} &= \text{diag}(e^{m^{(1)} - m^{(2)}}) \tilde{\mathbf{O}}^{(1)} + e^{\mathbf{S}^{(2)} - m^{(2)}} \mathbf{V}^{(2)} = e^{s^{(1)} - m} \mathbf{V}^{(1)} + e^{s^{(2)} - m} \mathbf{V}^{(2)} \\
 \mathbf{O}^{(2)} &= \text{diag}(\ell^{(2)})^{-1} \tilde{\mathbf{O}}^{(2)} = \mathbf{O}.
 \end{aligned}$$

We describe the full FLASHATTENTION-2 forward pass in Algorithm 1.

Algorithm 1 FLASHATTENTION-2 forward pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, block sizes B_c, B_r .

- 1: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
 - 2: Divide the output $\mathbf{O} \in \mathbb{R}^{N \times d}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, and divide the logsumexp L into T_r blocks $L_1, \dots, L_{T_r}$ of size B_r each.
 - 3: **for** $1 \leq i \leq T_r$ **do**
 - 4: Load $\mathbf{Q}_i$ from HBM to on-chip SRAM.
 - 5: On chip, initialize $\mathbf{O}_i^{(0)} = (0)_{B_r \times d} \in \mathbb{R}^{B_r \times d}, \ell_i^{(0)} = (0)_{B_r} \in \mathbb{R}^{B_r}, m_i^{(0)} = (-\infty)_{B_r} \in \mathbb{R}^{B_r}$.
 - 6: **for** $1 \leq j \leq T_c$ **do**
 - 7: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
 - 8: On chip, compute $\mathbf{S}_i^{(j)} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
 - 9: On chip, compute $m_i^{(j)} = \max(m_i^{(j-1)}, \text{rowmax}(\mathbf{S}_i^{(j)})) \in \mathbb{R}^{B_r}, \tilde{\mathbf{P}}_i^{(j)} = \exp(\mathbf{S}_i^{(j)} - m_i^{(j)}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\ell_i^{(j)} = e^{m_i^{(j-1)} - m_i^{(j)}} \ell_i^{(j-1)} + \text{rowsum}(\tilde{\mathbf{P}}_i^{(j)}) \in \mathbb{R}^{B_r}$.
 - 10: On chip, compute $\mathbf{O}_i^{(j)} = \text{diag}(e^{m_i^{(j-1)} - m_i^{(j)}}) \mathbf{O}_i^{(j-1)} + \tilde{\mathbf{P}}_i^{(j)} \mathbf{V}_j$.
 - 11: **end for**
 - 12: On chip, compute $\mathbf{O}_i = \text{diag}(\ell_i^{(T_c)})^{-1} \mathbf{O}_i^{(T_c)}$.
 - 13: On chip, compute $L_i = m_i^{(T_c)} + \log(\ell_i^{(T_c)})$.
 - 14: Write $\mathbf{O}_i$ to HBM as the i -th block of $\mathbf{O}$.
 - 15: Write L_i to HBM as the i -th block of L .
 - 16: **end for**
 - 17: Return the output $\mathbf{O}$ and the logsumexp L .
-

Causal masking. One common use case of attention is in auto-regressive language modeling, where we need to apply a causal mask to the attention matrix $\mathbf{S}$ (i.e., any entry $\mathbf{S}_{ij}$ with $j > i$ is set to float('inf')).

1. As FLASHATTENTION and FLASHATTENTION-2 already operate by blocks, for any blocks where all the column indices are more than the row indices (approximately half of the blocks for large sequence length), we can skip the computation of that block. This leads to around 1.7-1.8x speedup compared to attention without the causal mask.
2. We do not need to apply the causal mask for blocks whose row indices are guaranteed to be strictly less than the column indices. This means that for each row, we only need apply causal mask to 1 block (assuming square block).

Correctness, runtime, and memory requirement. As with FLASHATTENTION, Algorithm 1 returns the correct output $\mathbf{O} = \text{softmax}(\mathbf{Q}\mathbf{K}^T)\mathbf{V}$ (with no approximation), using $\mathcal{O}(N^2 d)$ FLOPs and requires $\mathcal{O}(N)$ additional

memory beyond inputs and output (to store the logsumexp L). The proof is almost the same as the proof of Dao et al. (2022, Theorem 1), so we omit it here

3.1.2 BACKWARD PASS (To recheck algo.)

The backward pass of FLASHATTENTION-2 is almost the same as that of FLASHATTENTION. We make a minor tweak to only use the row-wise logsumexp L instead of both the row-wise max and row-wise sum of exponentials in the softmax.

A FLASHATTENTION-2 BACKWARD PASS

Algorithm 2 FLASHATTENTION-2 Backward Pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}, \mathbf{dO} \in \mathbb{R}^{N \times d}$ in HBM, vector $L \in \mathbb{R}^N$ in HBM, block sizes B_c, B_r .

- 1: Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
- 2: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide $\mathbf{dO}$ into T_r blocks $\mathbf{dO}_1, \dots, \mathbf{dO}_{T_r}$ of size $B_r \times d$ each, and divide L into T_r blocks $L_1, \dots, L_{T_r}$ of size B_r each.
- 3: Initialize $\mathbf{dQ} = (0)_{N \times d}$ in HBM and divide it into T_r blocks $\mathbf{dQ}_1, \dots, \mathbf{dQ}_{T_r}$ of size $B_r \times d$ each. Divide $\mathbf{dK}, \mathbf{dV} \in \mathbb{R}^{N \times d}$ in to T_c blocks $\mathbf{dK}_1, \dots, \mathbf{dK}_{T_c}$ and $\mathbf{dV}_1, \dots, \mathbf{dV}_{T_c}$, of size $B_c \times d$ each.
- 4: Compute $D = \text{rowsum}(\mathbf{dO} \circ \mathbf{O}) \in \mathbb{R}^d$ (pointwise multiply), write D to HBM and divide it into T_r blocks $D_1, \dots, D_{T_r}$ of size B_r each.
- 5: **for** $1 \leq j \leq T_c$ **do**
- 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
- 7: Initialize $\mathbf{dK}_j = (0)_{B_c \times d}, \mathbf{dV}_j = (0)_{B_c \times d}$ on SRAM.
- 8: **for** $1 \leq i \leq T_r$ **do**
- 9: Load $\mathbf{Q}_i, \mathbf{O}_i, \mathbf{dO}_i, \mathbf{dQ}_i, L_i, D_i$ from HBM to on-chip SRAM.
- 10: On chip, compute $\mathbf{S}_i^{(j)} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 11: On chip, compute $\mathbf{P}_i^{(j)} = \exp(\mathbf{S}_i^{(j)} - L_i) \in \mathbb{R}^{B_r \times B_c}$.
- 12: On chip, compute $\mathbf{dV}_j \leftarrow \mathbf{dV}_j + (\mathbf{P}_i^{(j)})^T \mathbf{dO}_i \in \mathbb{R}^{B_c \times d}$.
- 13: On chip, compute $\mathbf{dP}_i^{(j)} = \mathbf{dO}_i \mathbf{V}_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 14: On chip, compute $\mathbf{dS}_i^{(j)} = \mathbf{P}_i^{(j)} \circ (\mathbf{dP}_i^{(j)} - D_i) \in \mathbb{R}^{B_r \times B_c}$.
- 15: Load $\mathbf{dQ}_i$ from HBM to SRAM, then on chip, update $\mathbf{dQ}_i \leftarrow \mathbf{dQ}_i + \mathbf{dS}_i^{(j)} \mathbf{K}_j \in \mathbb{R}^{B_r \times d}$, and write back to HBM.
- 16: On chip, compute $\mathbf{dK}_j \leftarrow \mathbf{dK}_j + \mathbf{dS}_i^{(j)T} \mathbf{Q}_i \in \mathbb{R}^{B_c \times d}$.
- 17: **end for**
- 18: Write $\mathbf{dK}_j, \mathbf{dV}_j$ to HBM.
- 19: **end for**
- 20: Return $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$.

Multi-query attention and grouped-query attention. Multi-query attention (MQA) (Shazeer, 2019) and grouped-query attention (GQA) (Ainslie et al., 2023) are variants of attention where multiple heads of query attend to the same head of key and value, in order to reduce the size of KV cache during inference. Instead of having to duplicate the key and value heads for the computation, we implicitly manipulate the indices into the head to perform the same computation. In the backward pass, we need to sum the gradients $\mathbf{dK}$ and $\mathbf{dV}$ across different heads that were implicitly duplicated.

3.2 PARALLELISM

The first version of FLASHATTENTION parallelizes over batch size and number of heads. We use 1 thread block to process one attention head, and there are overall batch size*number of heads thread blocks. Each thread

block is scheduled to run on a streaming multiprocessor (SM), and there are 108 of these SMs on an A100 GPU for example. This scheduling is efficient when this number is large (say ≥ 80), since we can effectively use almost all of the compute resources on the GPU.

In the case of long sequences (which usually means small batch sizes or small number of heads[$\leftarrow$ ---BUT WHY--- $\rightarrow$]), to make better use of the multiprocessors on the GPU, we now additionally parallelize over the sequence length dimension. This results in significant speedup for this regime.

Forward pass. We see that the outer loop (over sequence length) is embarrassingly parallel, and we schedule them on different thread blocks that do not need to communicate with each other. We also parallelize over the batch dimension and number of heads dimension, as done in FLASHATTENTION. The increased parallelism over sequence length helps improve occupancy (fraction of GPU resources being used) when the batch size and number of heads are small, leading to speedup in this case.

Backward pass. Notice that the only shared computation between different column blocks is in update dQ in Algorithm 2, where we need to load dQ_i from HBM to SRAM, then on chip, update $dQ_i \leftarrow dQ_i + dS_i^{(j)} K_j$, and write back to HBM. We thus parallelize over the sequence length dimension as well, and schedule 1 thread block for each column block of the backward pass. We use atomic adds to communicate between different thread blocks to update dQ .

We describe the parallelization scheme in Fig. 2.

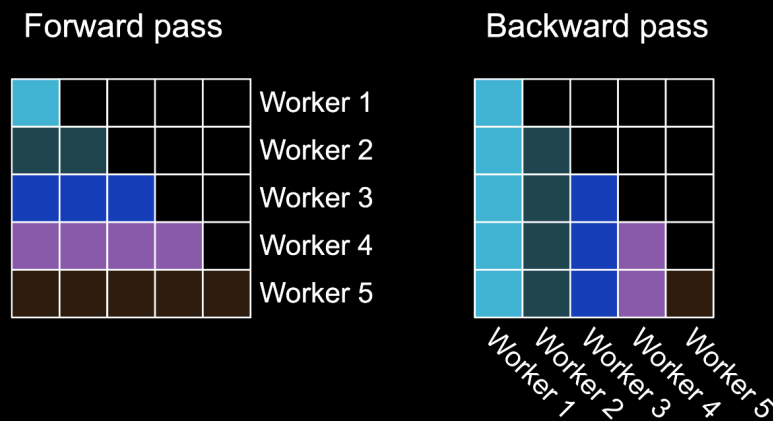


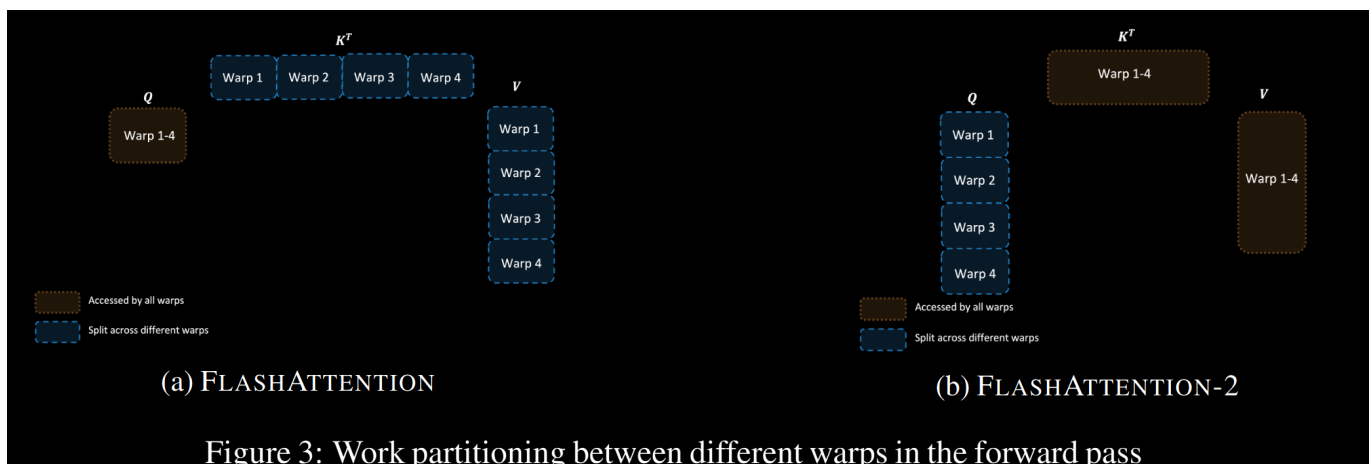
Figure 2: In the forward pass (left), we parallelize the workers (thread blocks) where each worker takes care of a block of rows of the attention matrix. In the backward pass (right), each worker takes care of a block of columns of the attention matrix.

Decoding. During LLM inference, most of the time is spent on iterative decoding, where one token is predicted at a time. The bottleneck for the attention operation during decoding is different from that during training or prefill (prompt processing), because the query length is very short (often query length is 1 since only the new extra token is attending to all the previous tokens, stored in the KV cache). As a result, the bottleneck is no longer the read/write of intermediate matrices the scores QK^T and attention probabilities $\text{softmax}(QK^T)$. Instead, the bottleneck is to load the KV cache as quickly as possible.

To accommodate this setting, we split the KV cache loading among different thread blocks, to increase occupancy and saturate the HBM bandwidth. However, since the thread blocks cannot easily communicate with each other, we write intermediate results to HBM, then call a separate kernel to reduce the results and produce final output.

3.3 WORK PARTITIONING BETWEEN WARPS As Section 3.2 describe how we schedule thread blocks, even within each thread block, we also have to decide how to partition the work between different warps. We typically use 4 or 8 warps per thread block, and the partitioning is described in Fig 3

Forward pass. For each block, FLASHATTENTION splits K and V across 4 warps while keeping Q accessible by all warps. Each warp multiplies to get a slice of QK^T , then they need to multiply with a slice of V and communicate to add up the result. This is referred to as the 'split-K' scheme. However, this is inefficient since all warps need to write their intermediate results out to shared memory, synchronize, then add up the intermediate results. These shared memory reads/writes slow down the forward pass in FLASHATTENTION. In FLASHATTENTION-2, we instead split Q across 4 warps while keeping K and V accessible by all warps. After each warp performs matrix multiply to get a slice of QK^T , they just need to multiply with their shared slice of V to get their corresponding slice of the output. There is no need for communication between warps. The reduction in shared memory reads/writes yields speedup



Backward pass. Similarly for the backward pass, we choose to partition the warps to avoid the "split-K" scheme. However, it still requires some synchronization due to the more complicated dependency between all the different inputs and gradients $Q \square K \square V \square O \square dO \square dQ \square dK \square dV$. Nevertheless, avoiding "split-K" reduces shared memory reads/writes and again yields speedup

Tuning block sizes Increasing block sizes generally reduces shared memory loads/stores, but increases the number of registers required and the total amount of shared memory. Past a certain block size, register spilling causes significant slowdown, or the amount of shared memory required is larger than what the GPU has available, and the kernel cannot run at all. Typically we choose blocks of size $\{64, 128\} \times \{64, 128\}$, depending on the head dimension d and the device shared memory size. We manually tune for each head dimensions since there are essentially only 4 choices for block sizes, but this could benefit from auto-tuning to avoid this manual labor. We leave this to future work.

4 EMPIRICAL VALIDATION

4.1 BENCHMARKING ATTENTION FOR TRAINING

Benchmark setting: we vary the sequence length from 512, 1k, ..., 16k, and set batch size so that the total number of tokens is 16k. We set hidden dimension to 2048, and head dimension to be either 64 or 128 (i.e., 32 heads or 16 heads). To calculate the FLOPs of the forward pass, we use:

$$4 \cdot \text{seqlen}^2 \cdot \text{head dimension} \cdot \text{number of heads}.$$

With causal mask, we divide this number by 2 to account for the fact that approximately only half of the entries are calculated. To get the FLOPs of the backward pass, we multiply the forward pass FLOPs by 2.5 (since there are 2 matmuls in the forward pass and 5 matmuls in the backward pass, due to recomputation)

4.2 BENCHMARKING ATTENTION FOR INFERENCE

from FasterTransformer. This is thanks to better work partitioning where multiple thread blocks are loading the KV cache at the same time to saturate HBM bandwidth.

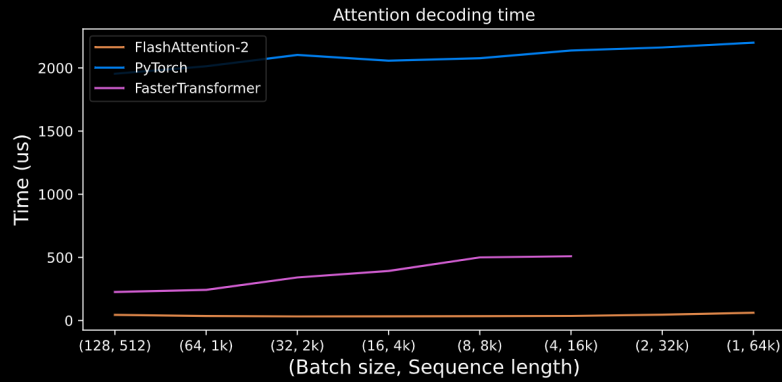


Figure 5: Attention decoding time on A100 80GB, with hidden dimension 2048 and multi-query attention. The attention kernel from FLASHATTENTION-2 is up to 7× faster than that of FasterTransformer and 28× faster than a naive implementation in PyTorch.

4.3 END-TO-END PERFORMANCE

Note that we calculate the FLOPs by the formula, following Megatron-LM (Shoeybi et al., 2019) (and many other papers and libraries): $6 * \text{seq_len} * \text{number of params} + 12 * \text{number of layers} * \text{hidden dim} * \text{seq_len}^2$. The first term accounts for the FLOPs due to weight-input multiplication, and the second term accounts for the FLOPs due to attention. However, one can argue that the second term should be halved, as with causal mask we only need to compute approximately half the number of elements in attention. We choose to follow the formula from the literature (without dividing the attention FLOPs by 2) for consistency.